

INSTITUTO TECNOLÓGICO DE JIQUILPAN

Ingeniería en Sistemas Computacionales



DOCUMENTO DE VISIÓN DEL PROYECTO Y ARQUITECTURA DE ALTO NIVEL

Sistema de Gestión de Flotillas con Monitoreo de Fatiga de Conductores

DESARROLLO DE APLICACIONES WEB

Docente: Martínez González Leonardo

Entregable 1 — Fundamentos y Planeación del Proyecto

Integrante	Rol
Angel de Jesús Canela Figueroa	Scrum Master / Product Owner
Isamar Chávez Díaz	Development Team
Katia Paola Victoria Escalera	Development Team

1. Problema y contexto

La fatiga del conductor es una de las causas de accidentes viales más difíciles de detectar a tiempo, porque sus señales (microsueños, bostezos prolongados) ocurren en segundos y dependen de la supervisión directa del propio conductor. Las empresas de transporte y logística que manejan flotillas medianas suelen gestionar vehículos, choferes y rutas en sistemas desconectados entre sí, y no cuentan con un mecanismo automático de alerta temprana ante señales de fatiga.

Este proyecto nace de la oportunidad de integrar un modelo de visión por computadora ya desarrollado y validado (Sistema de Detección de Fatiga en Conductores) dentro de un sistema completo de gestión de flotillas, de modo que la detección de fatiga deje de ser una herramienta aislada y se convierta en parte del flujo operativo real de una empresa de transporte.

2. Objetivos

Objetivo general

Diseñar y construir un sistema de gestión de flotillas basado en microservicios que integre un servicio de inteligencia artificial de detección de fatiga, permitiendo a los administradores de flotilla monitorear y responder a alertas de riesgo en tiempo real.

Objetivos específicos

- Gestionar de forma centralizada vehículos, choferes, rutas y viajes con integridad transaccional.
- Integrar el modelo existente de detección de fatiga como un microservicio de IA reutilizable.
- Generar alertas accionables (incidentes) a partir de episodios de fatiga detectados, evitando falsos positivos y garantizando que los episodios críticos siempre generen alerta.
- Desplegar el sistema completo de forma independiente por microservicio en Google Cloud Run.
- Aplicar la metodología Scrum durante las 16 semanas del proyecto con evidencia documentada en JIRA.

3. Alcance

Dentro del alcance

- Los 5 microservicios definidos (Identidad, Negocio, IA, y su integración vía Redis Streams), sus bases de datos, y el flujo de eventos completo.
- Autenticación JWT y control de acceso por rol (Entregable 3).
- Comunicación asíncrona IA → Negocio vía Redis Streams (Entregable 4).
- Despliegue en Google Cloud Run de los 5 microservicios (Entregable 5).

Fuera del alcance (trabajo futuro documentado)

- Geolocalización en tiempo real de vehículos.
- Módulo de pólizas de seguro.

- Procesamiento 100% en el borde (edge AI); se mantiene el envío de fotogramas vía API.
- Cumplimiento normativo formal de datos biométricos (aviso de privacidad legal); se documenta como consideración, no se implementa.

4. Equipo y roles Scrum

El Scrum Master se mantiene fijo durante todo el semestre, concentrado en la misma persona que funge como Product Owner, dada la organización interna del equipo.

Integrante	Rol
Angel de Jesús Canela Figueroa	Scrum Master y Product Owner (fijo)
Isamar Chávez Díaz	Development Team
Katia Paola Victoria Escalera	Development Team

5. Dominio y usuarios

Dominio elegido: gestión de flotillas de transporte con monitoreo de fatiga de conductores. Se reutiliza el proyecto de la materia de Visión por Computadora y Machine Learning (Sistema de Detección de Fatiga en Conductores) como servicio de IA del sistema.

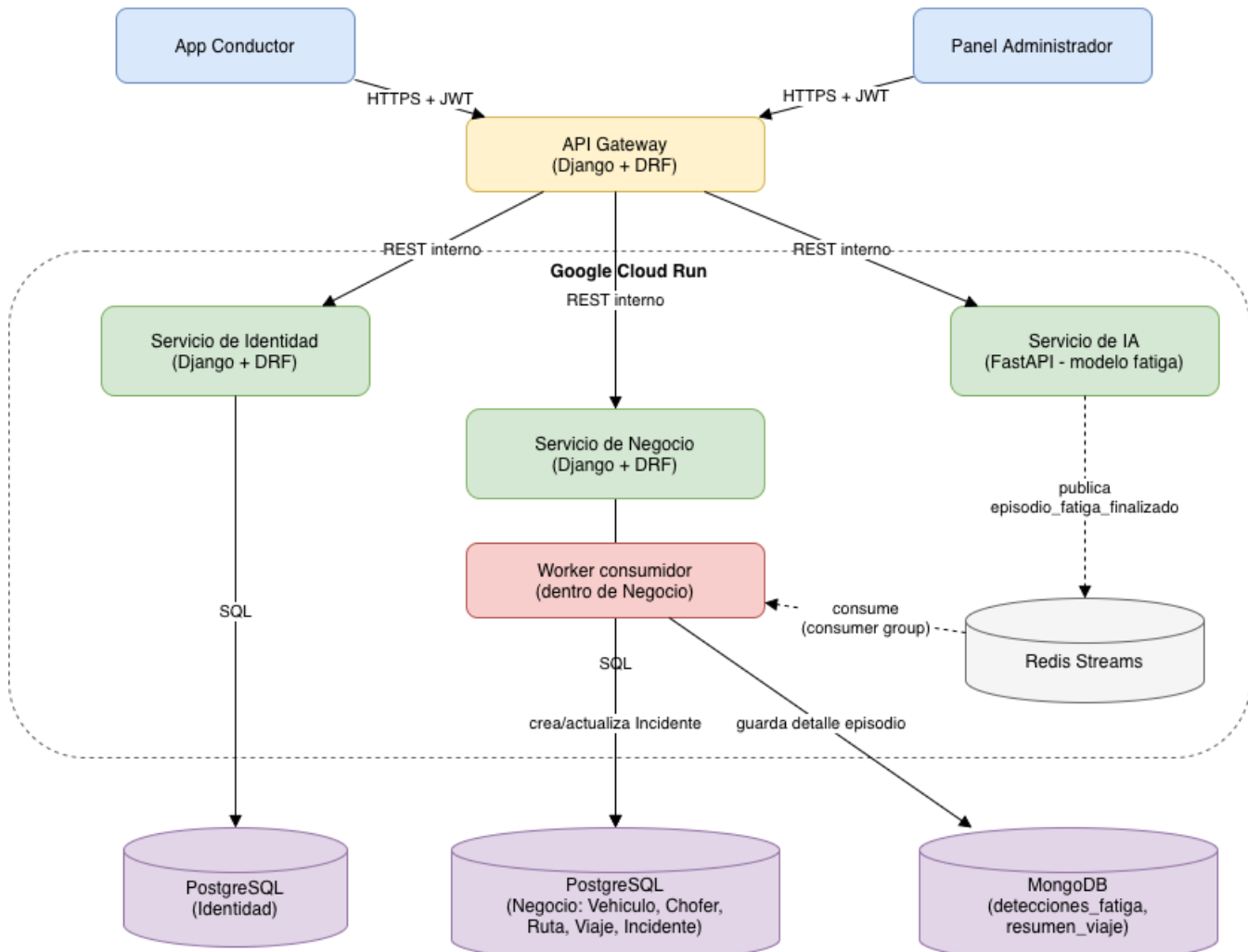
Tipos de usuario

- Administrador de flotilla: asigna choferes a vehículos, consulta alertas de fatiga, atiende o descarta incidentes, y da seguimiento a mantenimiento de vehículos.
- Conductor: su cámara alimenta el servicio de IA durante el viaje; puede consultar su propio historial de viajes y alertas.

6. Arquitectura de microservicios

El sistema sigue el esqueleto mínimo de microservicios solicitado: un API Gateway como punto de entrada único, un microservicio de Identidad, un microservicio de Negocio (con su worker consumidor), un microservicio de Integración de IA, Redis Streams como bróker de mensajería asíncrona, y despliegue independiente de cada servicio en Google Cloud Run.

Arquitectura de microservicios — Sistema de Gestión de Flotillas con Monitoreo de Fatiga — v1 — Entregable 1



línea sólida = sincrónico (REST/SQL) · línea punteada = asíncrono (evento) · cilindro = base de datos

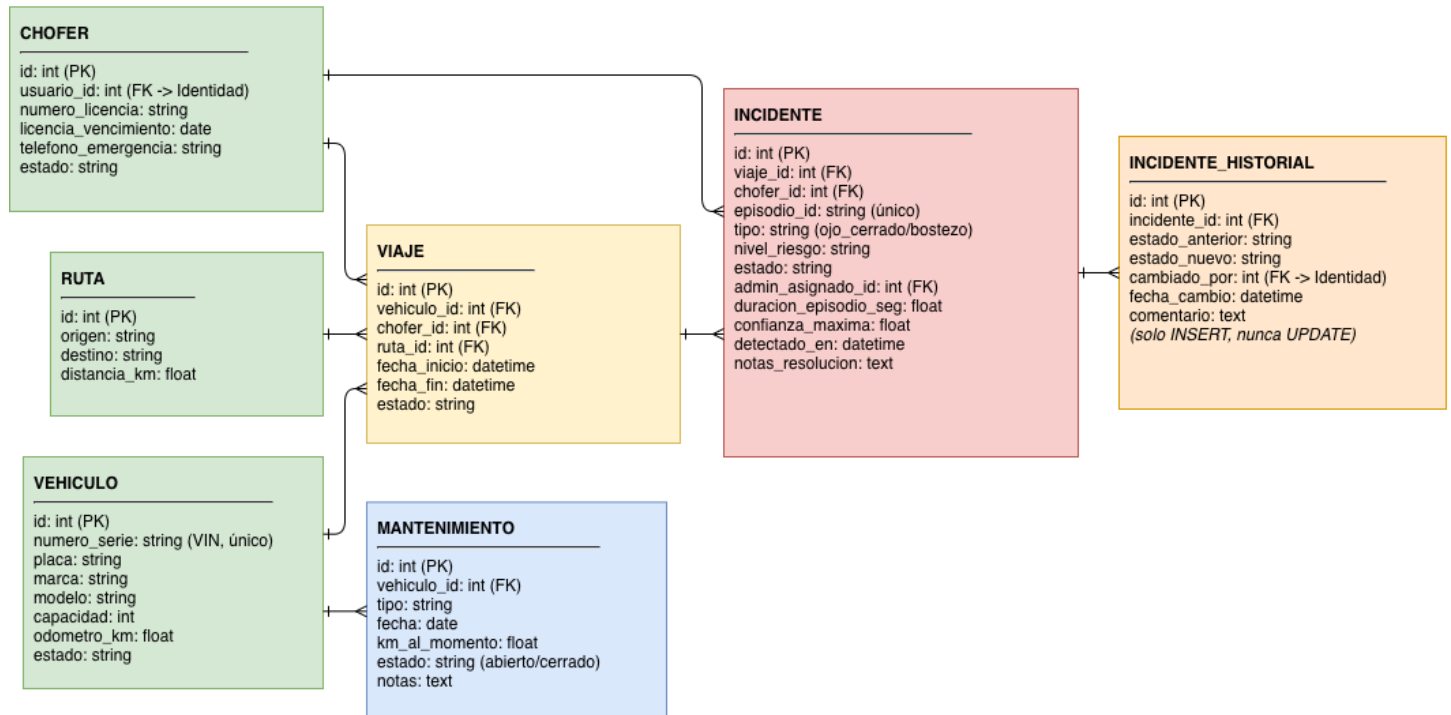
El flujo principal: la app del conductor envía fotogramas al Servicio de IA cada 800 ms. Cuando se detecta un episodio de riesgo sostenido (o este excede la duración máxima permitida de 8 segundos), el Servicio de IA publica un evento en Redis Streams. Un worker dentro del Servicio de Negocio consume ese evento, crea el registro de Incidente en PostgreSQL y guarda el detalle del episodio en MongoDB.

7. Diagrama entidad-relación (PostgreSQL)

El Servicio de Negocio mantiene 7 entidades relacionales: Vehiculo, Mantenimiento, Chofer, Ruta, Viaje, Incidente e Incidente_Historial. Las relaciones y atributos completos se muestran en el siguiente diagrama.

Diagrama ER — PostgreSQL (Servicio de Negocio) — v3 — Entregable 1

Nota: usuario_id, admin_asignado_id y cambiado_por son FK lógicas hacia el Servicio de Identidad — sin integridad referencial física entre microservicios, validadas a nivel aplicación. episodio_id es la única clave compartida con MongoDB (generada por el Servicio de IA antes de que exista el Incidente). Regla de negocio: un Chofer no puede tener dos Viaje en estado 'en_curso' simultáneamente; un episodio de fatiga se trunca a los 8 segundos si el riesgo persiste, para no depender de que el chofer 'despierte' para generar la alerta.



Nota: usuario_id, admin_asignado_id y cambiado_por son llaves foráneas lógicas hacia el Servicio de Identidad, sin integridad referencial física entre microservicios; se validan a nivel aplicación. episodio_id es la única clave compartida con MongoDB, generada por el Servicio de IA antes de que exista el Incidente correspondiente.

8. Modelo de documentos (MongoDB)

El microservicio de datos no estructurados mantiene dos colecciones: `detecciones_fatiga` (detalle interno de cada episodio de fatiga) y `resumen_viaje` (agregado analítico por viaje).

Colección `detecciones_fatiga` — documento de ejemplo

```
{
  "episodio_id": "12-88-1758897130",
  "chofer_id": 12,
  "viaje_id": 88,
  "timestamp_inicio": "2026-09-26T14:32:10Z",
  "timestamp_fin": "2026-09-26T14:32:13Z",
  "tipo": "ojo_cerrado",
  "duracion_episodio_seg": 3.2,
  "confianza_maxima": 0.94,
  "modelo_version": "mobilenetv2-v3",
  "truncado_por_duracion_maxima": false,
  "muestras": [
    { "timestamp": "2026-09-26T14:32:10.0Z", "clase": "ojo_cerrado", "confianza": 0.88 },
    { "timestamp": "2026-09-26T14:32:11.6Z", "clase": "ojo_cerrado", "confianza": 0.94 }
  ],
  "frame_referencia_url": null
}
```

Colección `resumen_viaje` — documento de ejemplo

```
{
  "viaje_id": 88,
  "chofer_id": 12,
  "fecha_generado": "2026-09-26T17:00:00Z",
  "total_episodios": 3,
  "episodios_criticos": 1,
  "distribucion": { "ojo_cerrado": 2, "bostezo": 1 },
  "score_riesgo_viaje": 0.62,
  "duracion_total_alertas_seg": 14.4
}
```

Regla de negocio: un episodio se trunca a un máximo de 8 segundos. Si el riesgo persiste más allá de ese umbral, se dispara una alerta crítica de inmediato, sin esperar a que el episodio termine de forma natural — esto evita que un microsueño prolongado se quede sin generar alerta.

9. Justificación de bases de datos

A continuación se justifica, entidad por entidad, por qué cada dato reside en PostgreSQL o en MongoDB.

#	Entidad / Colección	Base de datos	Justificación
1	Vehiculo	PostgreSQL	Catálogo estable con integridad referencial hacia Viaje y Mantenimiento; se consulta con JOINS frecuentes.
2	Mantenimiento	PostgreSQL	Depende de Vehiculo (FK); sostiene la regla de que un vehículo con mantenimiento abierto no puede recibir un viaje.
3	Chofer	PostgreSQL	Datos regulados (licencia) consistentes con Viaje e Incidente; no es de alto volumen ni esquema variable.
4	Ruta	PostgreSQL	Catálogo simple y estable, reutilizado por múltiples viajes.
5	Viaje	PostgreSQL	Núcleo transaccional del negocio; sostiene la regla de un solo viaje en_curso por chofer.
6	Incidente	PostgreSQL	Caso accionable del administrador; requiere estado transaccional confiable y JOINS constantes.
7	Incidente_Historial	PostgreSQL	Requiere integridad transaccional estricta con Incidente: el cambio de estado y su auditoría se guardan en la misma transacción.
8	detecciones_fatiga	MongoDB	Alto volumen, estructura interna variable (muestras, frame opcional), sin integridad transaccional con el negocio.
9	resumen_viaje	MongoDB	Agregado analítico recalculado de forma independiente; esquema flexible ante nuevas métricas.

10. Funcionalidad de inteligencia artificial

El servicio de IA reutiliza el modelo ya entrenado del proyecto de Detección de Fatiga en Conductores: una red MobileNetV2 con transfer learning en dos fases, entrenada sobre más de 54,000 imágenes de datasets públicos, con 99.02% de exactitud en pruebas. El modelo clasifica fotogramas en 4 clases (ojo_abierto, ojo_cerrado, bostezo, no_bostezo) y se expone como microservicio mediante FastAPI.

Dentro de este proyecto, el servicio recibe fotogramas en tiempo real desde la app del conductor cada 800 ms, agrupa las detecciones de riesgo sostenido en episodios (con un máximo de 8 segundos por episodio), y publica un evento a Redis Streams cuando un episodio se completa o cuando excede la duración máxima permitida, para su procesamiento asíncrono por el microservicio de Negocio.

11. Plan de sprints

Planeación general de los 6 sprints alineados a las 16 semanas y a las 5 entregas del semestre. El detalle de historias de usuario y story points se documenta en el proyecto JIRA del equipo.

Sprint	Semanas	Entrega asociada
Sprint 0	1-3	Entregable 1: Fundamentos y planeación
Sprint 1 y 2	4-7	Entregable 2: Microservicios (Core)
Sprint 3	8-9	Entregable 3: Seguridad y control de acceso
Sprint 4	10-12	Entregable 4: Comunicación asíncrona
Sprint 5 y 6	13-16	Entregable 5: Despliegue y cierre

12. Riesgos

Riesgo	Impacto	Mitigación
Curva de aprendizaje de Redis Streams	Medio	Prototipo aislado antes de integrarlo en el Sprint del Entregable 4.
Cold starts de Cloud Run en el Servicio de IA	Medio	Configurar instancias mínimas activas (min-instances) en producción.
Inconsistencia Postgres/Mongo por episodio_id duplicado	Alto	Índice único en ambas bases; reintentos idempotentes en el worker.
Manejo de datos biométricos sin marco legal formal	Bajo	Documentado como consideración de cumplimiento, sin implementarse en este entregable.
Carga de trabajo desigual entre semanas (exámenes, otras materias)	Medio	Revisar velocidad del equipo en cada retrospectiva y ajustar el backlog restante.